Lesson plan · for the teacher

Read before you write: PRIMM and tracing

Programming · Level 3 · Lesson 1

Time
40 min

Materials
A short program on the board or on paper; Trace table sheets (one column per variable); One phone to run the program

Learning goals

- Plan a PRIMM lesson.
- Trace a program in a trace table.
- Write predict questions that reveal misconceptions.

Activity: Predict, run, investigate

1. Predict (5 min): Write a four-line loop that adds marks on the board. Pairs write what it prints, without talking to other pairs.
2. Run (5 min): Run it on your phone and read the output aloud. Tally the predictions. Ask a pair who was wrong to explain first.
3. Trace (10 min): Pupils draw a trace table (a column for each variable, a row for each step) and fill it in. Check rows together.
4. Modify (15 min): Pairs change the program: count the marks of 10 or more, or print the average. Run a few on the phone.
5. Wrap-up (5 min): Ask which misconception the most common wrong prediction showed. Then give the exit ticket.

Only the Run stage needs a device, and one phone is enough. Everything else works on paper.

Discussion

- Why does predicting help pupils more than running straight away?
- Which misconception did the predictions show?
- How could you use trace tables without any devices?

Exit ticket

- Write a predict question for a three-line program, with one wrong answer and the misconception it shows.

Try it at school tomorrow
Start your next programming lesson with a program pupils can’t run yet: they predict on paper, then you run it on one phone. Ask the pupils who were wrong to explain their thinking first.

Practice questions · for you

Read before you write: PRIMM and tracing

Programming · Level 3 · Lesson 1

1 **In a PRIMM lesson, what do pupils do before they run the program?**

◯ Change one line ◯ Predict what it will do ◯ Write their own program

2 **Modify: this working program adds up a number of marks. Change it so it also prints the average on a second line, like Average: 12.0.**

```
n = int(input("How many marks? "))
total = 0
for i in range(n):
    mark = int(input("Mark? "))
    total = total + mark
print("Total:", total)
```

Write your program:

3 **A predict question for your class. What does this print?**

```
total = 0
for m in [3, 5, 2]:
    total = total + m
print(total)
```

◯ 10 ◯ 3, 8 and 10, one per line ◯ 0 ◯ 2

4 **Put the PRIMM stages in order.**

☐ Run it and compare

☐ Investigate: answer questions about it

☐ Predict what the program will do

☐ Make your own program

☐ Modify it to do something new

5 **In PRIMM, pupils run the program first and then explain what it did.**

◯ True ◯ False

✎ **Take a program you will teach next week. Write one predict question for it, and say which misconception each wrong answer would show.**

Check your answers

Read before you write: PRIMM and tracing

Programming · Level 3 · Lesson 1

1 Predict what it will do

Predict comes first: reading the code and committing to an answer.

2

```
n = int(input("How many marks? "))
total = 0
for i in range(n):
    mark = int(input("Mark? "))
    total = total + mark
print("Total:", total)
print("Average:", total / n)
```

Modifying a working program is a small, safe step between reading code and writing it from scratch: pupils change one thing and see the effect.

3 10

Each wrong answer is a misconception: 2 means "total holds only the last value"; three lines means "print runs on every turn" (missing the indentation); 0 means "the loop doesn't change total".

4 Top to bottom: 2, 3, 1, 5, 4

Reading comes before writing: pupils predict and run, explore how it works, change it, and only then write their own.

5 False

Pupils predict first. Committing to a prediction is what makes them read every line, and a wrong one shows you the misconception.